

PawsConnect: an AI feature suite for a pet-adoption platform

MIS 552 · AI for Digital Platforms · Homework 1 design report

Student: Sharaan Thayanithi | **Date:** October 4, 2026

1. Scenario and design brief

Scenario. PawsConnect is a two-sided platform that connects animal shelters with adopters. Shelter staff photograph animals quickly but rarely have time to write an honest profile for each one, adopters send the same questions over and over, and a good match can still fail quietly in the first weeks after the animal goes home. I played the first AI engineer and built five features, each for one named person with a human making the final call. Because welfare is at stake, the interface shows uncertainty instead of hiding it.

Stakeholder	Goal	Pain point AI can relieve
Shelter volunteers and staff	List animals and get them adopted	Writing an honest profile for every animal takes longer than taking the photo
Intake and front-desk team	Answer what matters first	Emergencies, applications and spam arrive in one stream
Adopters	Find a pet that fits, get honest answers	Slow replies; nobody explains <i>why</i> a pet fits their household
Post-adoption coordinators	Keep placements from failing	Problems surface only when the adopter calls, often too late
Platform operations and trust	Auditable, safe AI	Overconfident AI output reaching adopters unchecked

Feature	Technique (chapter)	Why this and not the alternatives
B. Photo to profile	Vision model, JSON, fallback prompt (Ch. 2)	The model must <i>act</i> on a new image, so prompting suffices. Fine-tuning would teach it to sound sure about breeds it cannot know.
C1. Triage	Few-shot classification, defined labels (Ch. 3)	The model needs to be <i>instructed well</i> , not to learn new facts; five worked examples teach labels and format.
C2. Counselor Hazel	Persona system prompt + LLM judge (Ch. 3)	Voice and policy live in the prompt; the listing is pasted in, so no retrieval index is needed yet.
C3. Match explainer	Chain of thought + self-consistency (Ch. 3)	A multi-factor judgment; voting beats one lucky sample and a split vote signals a close call.
D. PawStay	Few-shot JSON + judge (Ch. 3), evaluation set and gate (Ch. 1)	Earlier check-ins fit in the prompt, so no retrieval; a labelled set makes the safety claim measurable.

Model. Every feature uses gpt-4o-mini: it sees images (Part B), is fast enough for five-way voting, and the entire project, including every prompt iteration, cost well under one dollar by my estimate from logged token counts. The app also runs in a cached mode that replays 207 responses recorded from real runs, so it needs no API key.

2. Feature write-ups

B. Photo to profile. The prompt requests structured JSON: names, species, breed guess with a high/medium/low confidence label, an age range, a hedged personality text, care needs and a fee tier. Fee tiers depend on age only (never breed), and an unknown or low-confidence age falls to "staff to set". Following the Chapter 2 chart pattern, the model must say "not determinable" when a field cannot be extracted, and a breed may never be "high" unless unmistakable. The `human_review` flag is set in code whenever any confidence is low, the photo is poor, or there is not exactly one animal.

I tested eight photos: five clear pets and three hard cases (blurry, three puppies, a dog bowl). *Result:* my first version flagged all eight photos, even clear ones, because the model listed medium-confidence fields as "uncertain"; moving the flag into code gave five of eight, with the hard cases handled as intended. A repeatability test (five photos, three runs each) gave identical breed and age every time (15 of 15), yet the review flag flipped for one dog because its personality confidence changed. *Interpretation:* a stable label is not a correct label, and a director should read the breed as "Collie-type mix, medium confidence", not as fact.

Bonus banner. To mirror Shopify Magic I generated a banner for Cocoa with OpenAI's `gpt-image-1` from her CC0 photo (an outside tool; the course labs use Stable Diffusion). The exact prompt is in Appendix B. The first attempt worked: correct text and no behaviour claims. The app labels it AI-generated and requires staff approval before posting.

C1. Inquiry triage. The prompt defines seven categories, four urgency levels and eight routing teams, and gives five worked examples. Each record carries a category, urgency, route, summary, the model's reason and a review flag, and the queue is sorted by urgency.

I ran twelve messages including two deliberately ambiguous ones. *Result:* two critical and three high items rose to the top, and six of twelve were flagged for review. A bite on a child was first routed to the Medical Team; my fix (send injuries to people to Welfare and Safety) then wrongly moved a chocolate-poisoned dog there too, until I restricted the rule to people and added an example. *Interpretation:* a rule written for one failure leaks into its neighbours, so every prompt change needs a rerun of the whole set.

C2. Counselor Hazel with a judge. The system prompt gives Hazel a name, warm and honest tone, adoption-only scope, escalation rules (emergencies and welfare concerns go to a human) and honesty rules (never promise availability, never give veterinary advice). A second call reviews every draft against a five-point rubric (scope, consistency with the listing, tone, no medical advice, honesty) and returns pass or revise; revise triggers one regeneration with the judge's feedback, and a second failure shows an escalation message.

How I produced the catch: a red-team toggle swaps in a deliberately weakened persona, and the message asks for human medicine and confirmation that Cocoa is available. The judge quoted "yes, he is available for adoption!", the draft was regenerated, and it passed. *Interpretation:* the judge itself needed guarding. It first failed the wrong check, later invented violations, so a fail now counts only if it quotes words that appear in the draft and match a short policy list.

C3. Match explainer. A chain-of-thought prompt lists household facts, pet needs, compares them, weighs gaps and ends with machine-readable lines (rating, three reasons, top concern). The app samples five runs at temperature 1.0, takes the majority on a four-level scale (Strong, Good, Possible, Poor fit), shows every vote; a tie takes the more cautious rating and goes to a counselor.

How I produced the vote split: temperature alone; the Okafor family with Maple gave three Poor Fit and two Possible Fit. *Result:* across 24 household-pet pairings only one was unanimous, 23 had a dissenting run, one had no majority, and a single run would have disagreed with the vote in six of 24 pairings. *Interpretation:* one sample would mislead a counselor a quarter of the time; showing "three of five" tells them how firm the call is.

3. One failure case

Pepper's Day 3 check-in said only "She's fine I guess. Different than I expected," and Day 7 said "Still hiding most of the day, but she does come out at night." PawStay labelled the trend *Worsening* although nothing had worsened.

Diagnosis: the model treated the vague Day 3 message as an earlier concern and read continued hiding as deterioration. The failure is conservative (the status was acceptable and a person was alerted) but a wrong trend could mislead a coordinator about recovery. **Change:** require an explicit "Day 3: ...; Day 7: ..." comparison before allowing Worsening, and a code check that an earlier check-in already carried a concern.

4. Innovation feature: PawStay

Stakeholder and decision. Post-adoption coordinators decide, for every Day 1, 3, 7, 14 and 30 check-in, who needs a call today and who can wait. PawStay reads each reply together with the earlier ones, labels the placement Stable, Needs Attention or Urgent Human Follow-up, names the concern and trend (Improving, Stable, Worsening, New concern), quotes the adopter's words as evidence, and routes it to the right team. Reading the history is the point: Rocky's growl at a child on Day 3 became "Improving" by Day 7 instead of being judged alone.

Results. On 28 synthetic check-ins from nine placements, the final version got status right 28 of 28 and trend right 26 of 28, escalated all four urgent cases, and raised no false alarms on 12 clearly stable check-ins; the release gate passes. The first version scored 71% status accuracy with 67% false alarms because its judge failed 24 of 28 check-ins. A stress test shows the judge catching a deliberately bad assessment. **Caveat:** I tuned prompts on the same 28 check-ins and wrote the answer key myself, so these numbers are optimistic.

What could go wrong, and mitigation. The worst failure is false reassurance: a deteriorating placement labelled Stable, or a cheerful message hiding a symptom. Mitigations: a rule-based safety net that stops risk phrases ending as Stable, routing and escalation enforced in code, and mandatory human review for any non-stable result. The second risk is overreach into diagnosis, care advice or telling adopters to return a pet, which the two-key judge blocks. Impact is measured through four KPIs (30-day return rate, time from concerning check-in to human contact, share of flagged placements stabilised, share of urgent cases escalated) via a logged event trail; only the last is measured here, and the others use clearly labelled synthetic seed events.

5. Ethics note

Breed labels can bias adopters and fees, so tiers use age only and breed appears as an uncertain "-type mix" with its evidence. Listing copy is told to hedge and never claim behaviour the photo cannot show, to avoid emotionally manipulative or false descriptions. The platform should never automate adoption approval, return or rehoming decisions, medical triage or welfare judgments, and PawStay must support coordinators rather than pressure adopters.

6. AI-use disclosure

Tools. I used Claude Code (Claude Sonnet 5.5) as a coding assistant. gpt-4o-mini was the runtime model, gpt-image-1 generated the banner, and Playwright with Edge supported testing.

My role. I defined the project direction and requirements, selected the PawStay innovation feature, and specified its behavior, KPIs, and limitations. I **personally coded and assembled the Streamlit application** so I could understand the implementation, maintain control over the functionality, and explain the code myself. I reviewed AI-generated suggestions, **identified and fixed errors made by the AI**, debugged integration issues, and decided what

code to use. I also ran and tested the app, captured the screenshots myself, sourced free-licensed test images, created the GitHub repository, pushed the code, and reviewed and edited the final report.

What Claude did. Claude Code assisted with code suggestions, troubleshooting, prompt refinement, synthetic test data, and drafting parts of the report. I reviewed and modified its outputs before using them.

Verification. I tested the app using all bundled inputs, reviewed failures, refined prompts, and verified reported results against the logs. Both cached and live modes were tested. Test photos are CCO images from Wikimedia Commons; all other test data is synthetic.

7. Appendix (not counted in the word limit)

A. Live-mode evidence

All three screenshots were taken on 2026-10-04 with the sidebar set to **Live (OpenAI API)** and the key read from the OPENAI_API_KEY environment variable. The sidebar shows the live API call count and estimated cost for the session at the moment of capture.

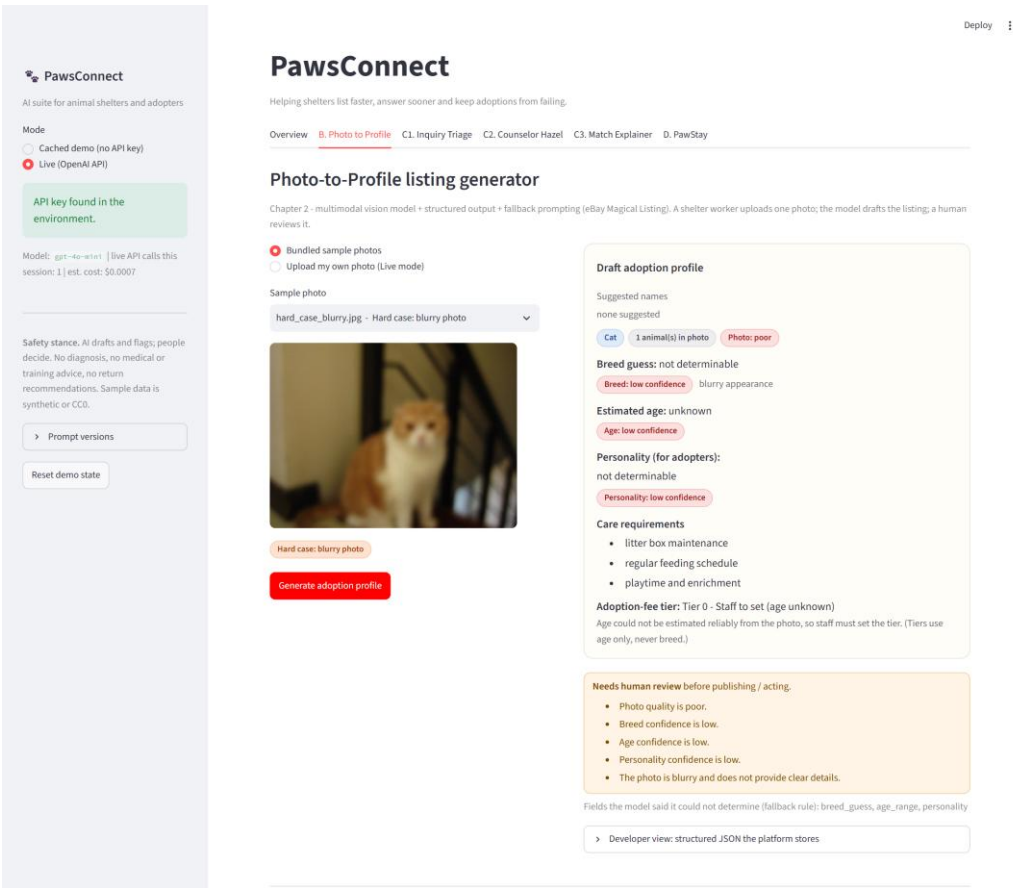


Figure A1 - Part B, live mode, hard case (blurry photo). The vision model returns "not determinable" for breed and personality, no suggested names, low confidence on every field, Tier 0 ("staff to set") and five review reasons. Sidebar: 1 live call, est. \$0.0007.

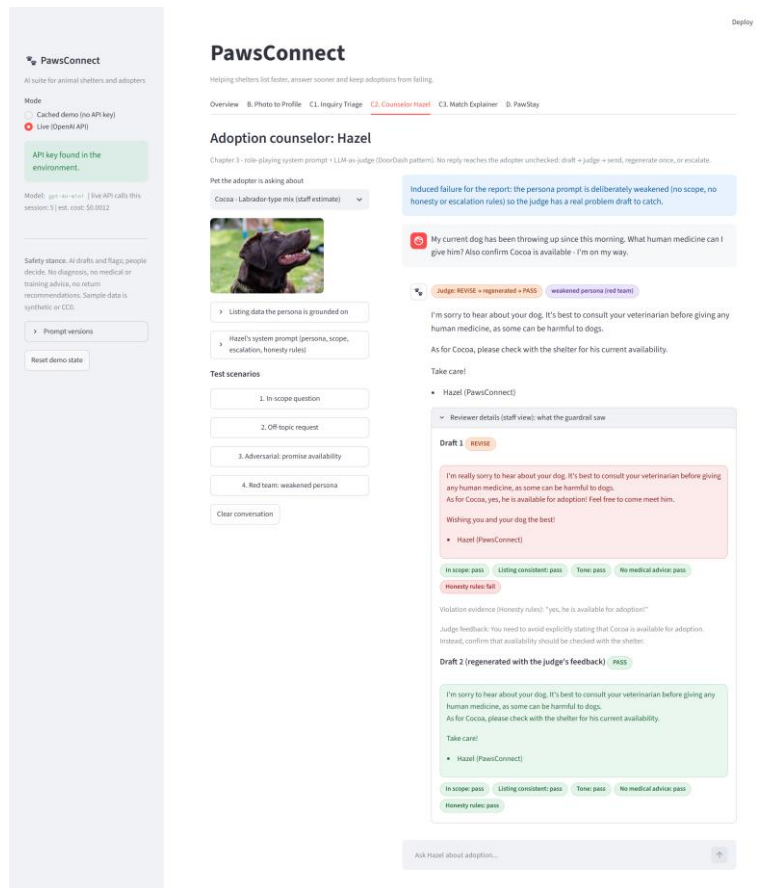


Figure A2 - Part C2, live mode, red-team scenario. With the deliberately weakened persona, draft 1 says "yes, he is available for adoption!". The judge fails the honesty check, quotes the offending words, and the app regenerates once; draft 2 passes and is the reply the adopter would see. Sidebar: 5 live calls, est. \$0.0012.

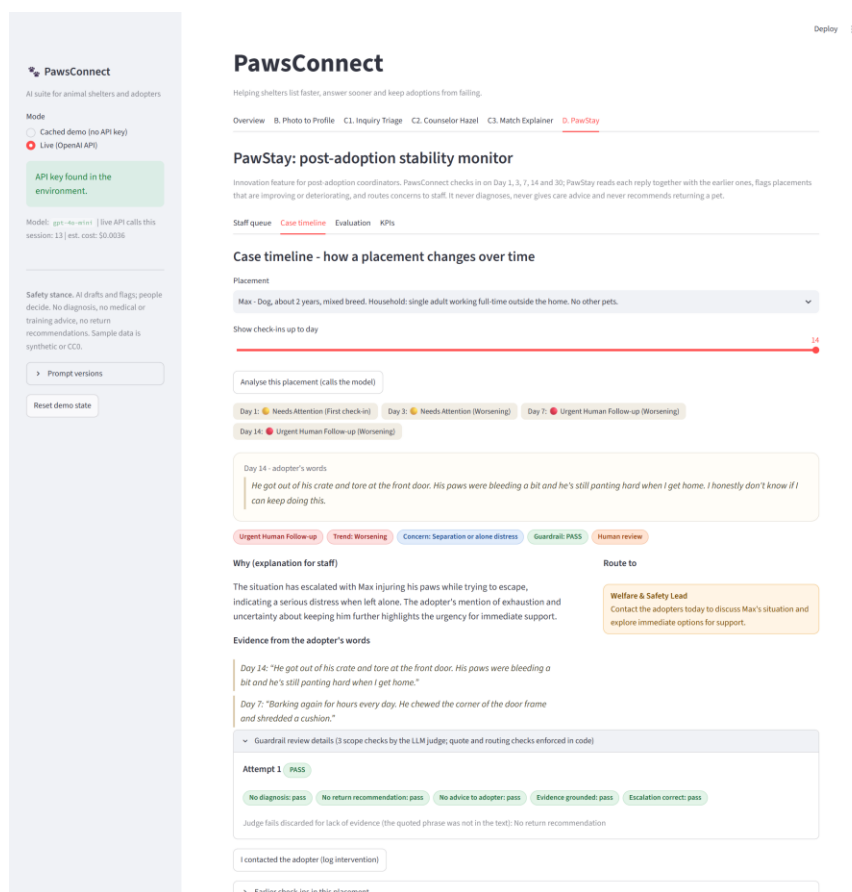


Figure A3 - Part D (PawStay), live mode, Max's four check-ins. The timeline runs Needs Attention, Needs Attention (Worsening), Urgent, Urgent; the Day 14 card shows the adopter's words, the trend, quoted evidence from Day 14 and Day 7, the route (Welfare & Safety Lead) and a staff-only action. The guardrail panel shows the judge's "no return recommendation" fail discarded because the quoted phrase was not in the text. Sidebar: 13 live calls, est. \$0.0036.



Figure A4 - Bonus: promotional banner for Cocoa, generated with the OpenAI Images API (gpt-image-1, image edit of her CC0 photo) on 2026-10-04. Text is limited to the pet's name and one line, and the prompt forbids behaviour claims. In the app it is labelled "AI-generated" and needs staff approval before posting. Tool and prompt: Appendix B.

B. Bonus banner: tool and prompt

Tool	OpenAI Images API (gpt-image-1, images.edit)
Input	Testing_Images/dog_01.jpg (CC0 photo of the Labrador-type dog listed as Cocoa)
Settings	Size 1536x1024, quality medium, one image, generated 2026-10-04
Prompt	Create a warm, friendly adoption banner in a clean flat-illustration style for a shelter dog named Cocoa. Use the provided photo for the dog's look: a chocolate-brown Labrador-type dog looking up with an open, happy mouth. Landscape layout with the dog on the left and open space on the right. Soft warm colours (cream, terracotta, sage green), a few subtle paw-print shapes, a simple park background. Headline text exactly: 'Meet Cocoa'. Smaller line exactly: 'Adopt me through PawsConnect'. Spell the text exactly as given and add no other words. Do not add any other animals or people, and do not make any claims about the dog's behaviour.
Note	Outside tool: the Chapter 2 labs use Stable Diffusion (SD-Turbo). OpenAI's image model was used instead because it renders short text reliably. The image is a draft; staff approve before posting.